

Password Strength Checker

Completed Project Instructions

Python | Regex | Validation | Testing | Refactoring | Documentation

Project Status

COMPLETE

PURPOSE OF THIS DOCUMENT

This version contains the project instructions only. All development, testing, refactoring, README updates, and final technical-report work have been completed, so there are no blank worksheets or fill-in pages.

Completion Summary

- ☒ Core password checker implemented
- ☒ Password-specific suggestions added
- ☒ Code refactored and cleaned up
- ☒ 24 test cases completed
- ☒ Post-refactor retesting completed
- ☒ README finalized
- ☒ Final technical report completed

1. Project Goal and Requirements

What the application was required to accomplish.

Project Goal

Build a Python command-line Password Strength Checker that accepts a test password, validates it against a defined password policy, calculates how many character requirements are met, assigns a Weak, Medium, or Strong rating, and provides suggestions for missing requirements.

Required Validation Checks

- ☒ Minimum password length: 8 characters.
- ☒ At least one uppercase letter.
- ☒ At least one lowercase letter.
- ☒ At least one numeric character.
- ☒ At least one approved special character.

Approved Special Characters

! @ # \$ % & _ - * +

Required Output

- ☒ Display whether each password requirement passed or failed.
- ☒ Display a suggestion when a requirement is not met.
- ☒ Display the number of character requirements met out of 4.
- ☒ Display the final password-strength classification.

2. Final Password Policy

The exact classification rules used by the completed program.

Character Score

The four character-based checks - uppercase, lowercase, number, and approved special character - are stored as Boolean values and added together to create a character score from 0 to 4. The minimum-length requirement is evaluated separately.

STRONG

Minimum length is met AND all 4 character checks are met.

MEDIUM

Minimum length is met AND exactly 3 out of 4 character checks are met.

WEAK

The password does not meet the requirements for Strong or Medium. A password is still Weak if it fails the minimum-length requirement, even when all 4 character checks pass.

Policy Checklist

- ☒ Minimum length remains separate from the character score.
- ☒ Only approved special characters satisfy the special-character requirement.
- ☒ Spaces may count toward length but do not count as approved special characters.
- ☒ No maximum password length is currently enforced.

3. Development Instructions - Completed

The original build sequence, with all required work marked complete.

Phase 1 - Setup and Input

- ☒ Import Python's re module.
- ☒ Create the console header.
- ☒ Set the minimum password length.
- ☒ Create the approved special-character regex pattern.
- ☒ Prompt the user to enter a test password.

Phase 2 - Validation Checks

- ☒ Check password length with len().
- ☒ Use re.search() for uppercase, lowercase, numeric, and special-character detection.
- ☒ Convert regex search results to Boolean values.
- ☒ Print pass/fail messages and improvement suggestions.

Phase 3 - Scoring and Classification

- ☒ Use sum() on the four Boolean character results.
- ☒ Apply the Strong, Medium, and Weak policy rules.
- ☒ Print the character score and final strength rating.

4. Refactoring and Quality Requirements - Completed

Code cleanup performed after the baseline program worked correctly.

Refactoring Checklist

- ☒ Remove redundant passed/failed variables.
- ☒ Store regex-search results as Boolean values.
- ☒ Use clearer variable names for validation checks and regex patterns.
- ☒ Reuse stored Boolean results in `character_score`.
- ☒ Remove outdated debugging comments.
- ☒ Improve console wording and formatting.
- ☒ Preserve the original program behavior while cleaning the code.

Important Troubleshooting Lessons

- A non-empty regex pattern is truthy by itself; the program must test the actual `re.search()` result.
- `re.search()` returns a Match object or None, so results should be converted to Boolean values before scoring.
- The hyphen requires careful handling inside a regex character class.
- Special-character policy and regex behavior should be tested with both approved and unapproved symbols.

REFACTORING RESULT

The final refactor improved readability and consistency without changing the intended password classifications or validation behavior.

5. Testing Requirements - Completed

Manual validation and regression testing for the finished application.

Testing Checklist

- ☒ Minimum-length pass and fail cases.
- ☒ Uppercase present and missing.
- ☒ Lowercase present and missing.
- ☒ Number present and missing.
- ☒ Approved special character present and missing.
- ☒ Hyphen and asterisk regex edge cases.
- ☒ Unapproved special character handling.
- ☒ Strong classification.
- ☒ All Medium combinations with one missing character requirement.
- ☒ Weak classifications with two, one, or zero character requirements.
- ☒ Below-minimum-length password with all four character types.
- ☒ Empty input.
- ☒ Password containing spaces.
- ☒ Very long password.
- ☒ Improvement suggestions for failed requirements.
- ☒ Retesting after refactoring.

FINAL TEST STATUS

A total of 24 documented test cases were completed. The application was retested after refactoring to confirm that cleanup changes did not alter the intended behavior.

6. Documentation and Repository Requirements - Completed

Final project files and professional documentation.

Documentation Checklist

- ✓ README includes overview, security need, objectives, features, policy, technologies, run instructions, testing, security considerations, lessons learned, and future improvements.
- ✓ Test-case documentation includes expected results, actual results, status, evidence paths, refactoring notes, and retesting notes.
- ✓ Initial test evidence is retained.
- ✓ Post-refactor retest evidence is retained.
- ✓ Final technical report is completed.
- ✓ Real account passwords are not used in screenshots, source code, or testing documentation.

Recommended Repository Structure

```
password_strength_checker/  
|-- password_checker.py  
|-- README.md  
|-- test_cases/  
|   |-- test_cases.md  
|   `-- Retest cases/  
|-- test_cases_Initial Test Cases/  
`-- docs/  
    |-- Password_Strength_Checker_Completed_Project_Instructions.pdf  
    `-- Password_Strength_Checker_Final_Technical_Report.pdf
```

PROJECT COMPLETION

The Password Strength Checker has completed the required development, testing, refactoring, documentation, and final-report stages. Any additional work can now be treated as a future version or a separate learning exercise.